

Quantum Algorithms

From Qubits to Shor's Algorithm — and Beyond

A 1-hour introduction for undergraduates
No prior quantum computing experience required

Today's Roadmap

Part I — Foundations (20 min)

1. Why quantum computing?
2. Qubits & superposition
3. Quantum gates & entanglement
4. Deutsch–Jozsa (warm-up)

Part II — The Big Algorithms (20 min)

5. Grover's search algorithm
6. Shor's factoring algorithm

Part III — The NISQ Era (20 min)

7. Hamiltonians & variational methods
8. VQE, QAOA & error mitigation

Part I: Foundations

Qubits, Gates, and Entanglement

Why Quantum Computing?

Classical computers struggle with:

- Factoring large numbers
- Searching unsorted databases
- Simulating molecules
- Optimization problems

These problems have **exponential** cost classically.

Quantum computers exploit:

- **Superposition** — be in multiple states at once
- **Entanglement** — correlations with no classical analogue
- **Interference** — cancel wrong answers, amplify right ones

Result: **exponential** or **quadratic** speedups for specific problems.

Classical Bits vs. Qubits

Classical Bit

A bit is either 0 or 1.

Like a coin lying on a table:
heads **or** tails.



Qubit

A qubit can be in a **superposition**:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$

$$\text{where } |\alpha|^2 + |\beta|^2 = 1$$

Like a spinning coin:
both heads **and** tails until you look.

When measured $\rightarrow$ collapses to $|0\rangle$ or $|1\rangle$.

Quick Notation Guide

Symbol	Meaning
$ 0\rangle$	Qubit in state 0 (like classical 0)
$ 1\rangle$	Qubit in state 1 (like classical 1)
$ +\rangle = \frac{1}{\sqrt{2}}(0\rangle + 1\rangle)$	Equal superposition — 50/50 chance of 0 or 1
$ -\rangle = \frac{1}{\sqrt{2}}(0\rangle - 1\rangle)$	Equal superposition with a phase flip
$ \psi\rangle = \alpha 0\rangle + \beta 1\rangle$	General single-qubit state
$ 00\rangle, 01\rangle, 10\rangle, 11\rangle$	Two-qubit basis states

Think of $|\rangle$ (“ket”) as a labeled box holding a quantum state.

Quantum Gates

Classical logic has AND, OR, NOT. Quantum computing has unitary gates.

X Gate (quantum NOT)

$$X|0\rangle = |1\rangle$$

$$X|1\rangle = |0\rangle$$

Flips the qubit.

H Gate (Hadamard)

$$H|0\rangle = |+\rangle$$

$$H|1\rangle = |-\rangle$$

Creates superposition.

CNOT Gate (controlled-NOT)

Flips target qubit
only if control = $|1\rangle$.

Creates entanglement.

Key constraint: all quantum gates are reversible (unitary). No information is lost.

Entanglement

Apply H then CNOT to two qubits both starting in $|0\rangle$:

$$|00\rangle \xrightarrow{H} \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)|0\rangle \xrightarrow{\text{CNOT}} \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$$

This is a **Bell state** — measuring one qubit instantly determines the other, regardless of distance.

- Entangled qubits are **correlated** beyond any classical explanation
- Not “communication” — no information travels faster than light
- It is a resource that quantum algorithms consume to gain speedups

Warm-up: The Deutsch–Jozsa Algorithm

The Problem:

Given a function $f : \{0, 1\}^n \rightarrow \{0, 1\}$,
promised to be either:

- **Constant:** same output for all inputs
- **Balanced:** outputs 0 for half, 1 for half

Determine which one.

Classically: need up to $2^{n-1} + 1$ queries.

Quantumly: **1 query!**

How it works:

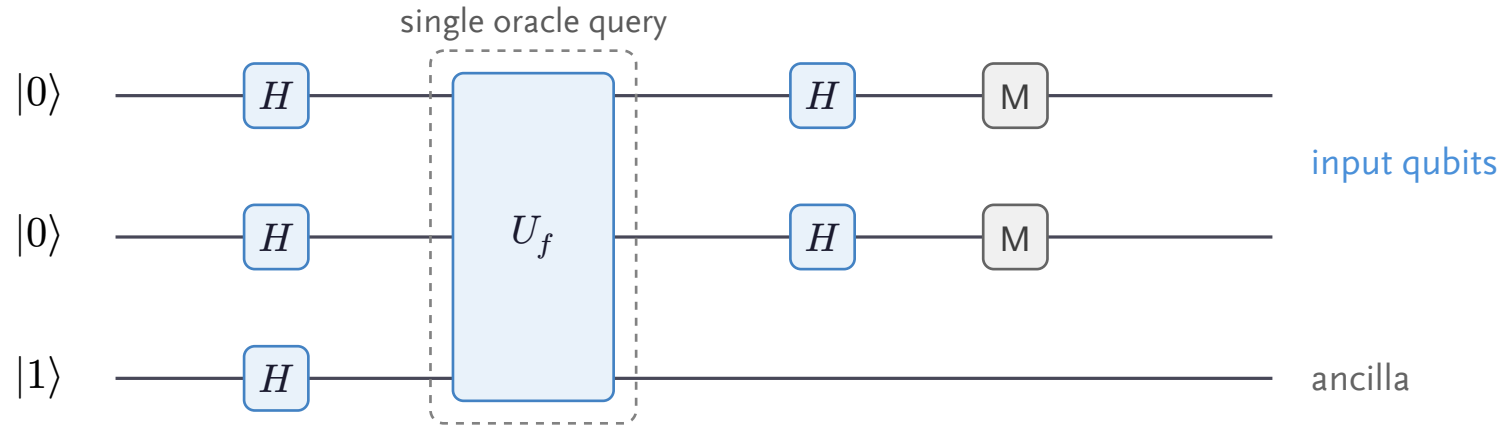
1. Put all n qubits in superposition with H
2. Query f once via quantum oracle U_f
3. Apply H again to input qubits
4. Measure

If constant $\rightarrow$ all zeros

If balanced $\rightarrow$ at least one non-zero

The speedup comes from interference:
wrong answers cancel out.

Deutsch–Jozsa: Circuit — simplified, $n = 2$ input qubits



The ancilla qubit in $|1\rangle$ enables **phase kickback** — the oracle marks solutions by flipping phases rather than flipping bits, which is what makes interference work on the input register.

Part II: The Big Algorithms

Grover's Search & Shor's Factoring

Grover's Search Algorithm

Lov Grover, 1996

Problem: Search an unsorted database of N items for a marked item.

Classical

Check items one by one.
On average: $N/2$ checks.
Worst case: N checks.
Complexity: $O(N)$

Quantum (Grover's)

Uses superposition + interference.
Needs only $\approx \sqrt{N}$ queries.
Complexity: $O(\sqrt{N})$

Quadratic speedup!

For $N = 1,000,000$: classical needs 500,000 steps, Grover's needs 1,000.

Grover's: How It Works

Two key operations, repeated $\approx \sqrt{N}$ times:

Step 1: Oracle

“Marks” the target item by flipping its amplitude’s sign.

$$|x\rangle \rightarrow \begin{cases} -|x\rangle & \text{if } x = \text{target} \\ |x\rangle & \text{otherwise} \end{cases}$$

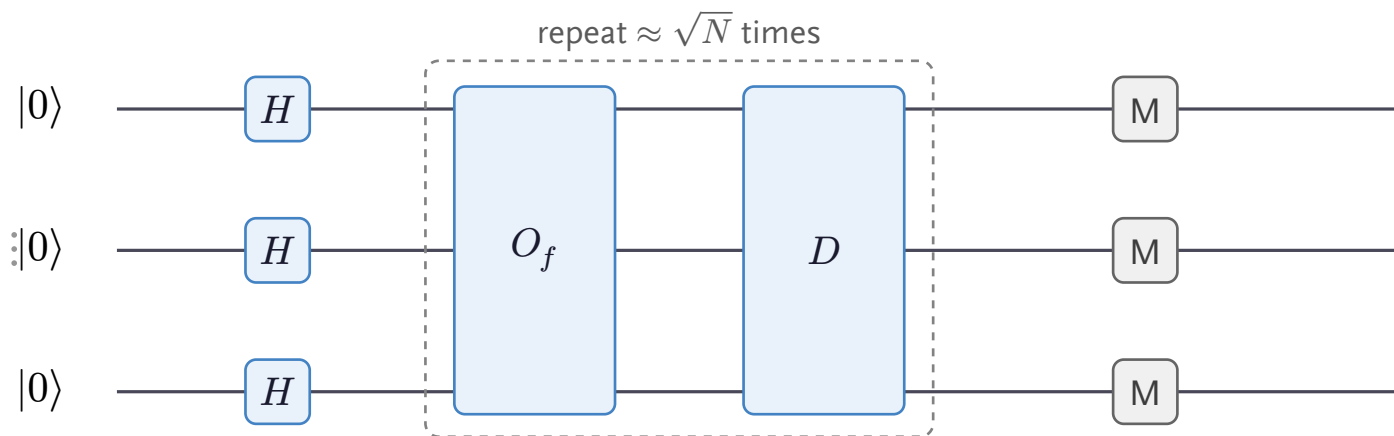
Step 2: Diffusion (Amplitude Amplification)

“Reflects” all amplitudes about their mean.

Below mean $\rightarrow$ pushed up
Above mean $\rightarrow$ pushed down
Net effect: target amplitude grows.

Start $\rightarrow$ uniform superposition $\rightarrow$ oracle $\rightarrow$ diffusion $\rightarrow$ repeat $\rightarrow$ measure

Grover's: Circuit — n -qubit search



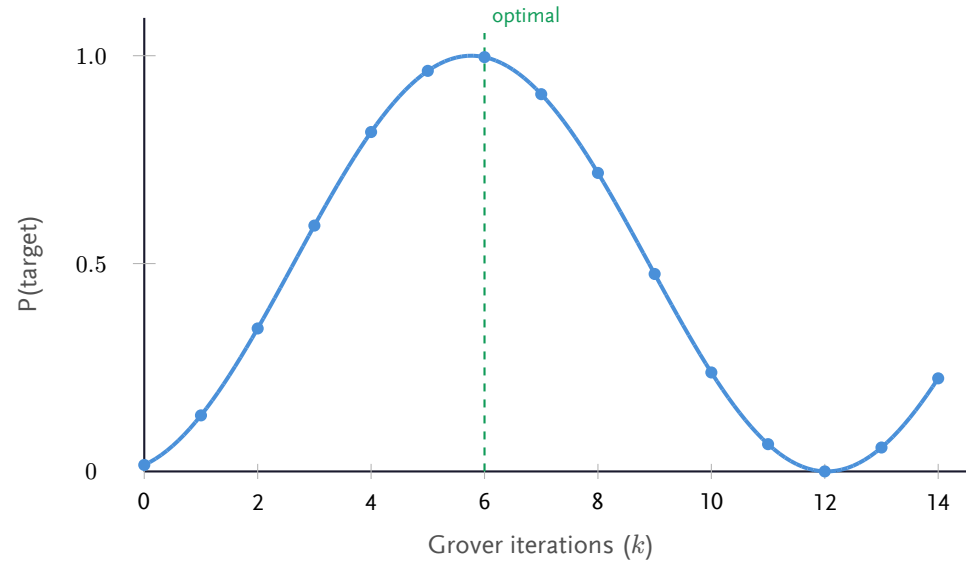
$H^{\otimes n}$: creates uniform superposition over all $N = 2^n$ states

O_f : oracle — flips the phase of the target state

$D = H^{\otimes n}(2|0\rangle\langle 0| - I)H^{\otimes n}$
diffusion — reflects about mean

Grover's: Amplitude Amplification

Target probability vs. Grover iterations ($N = 64$, optimal at ≈ 6 iterations)



Key observations:

- Probability of finding the target **oscillates**
- Peaks at $k \approx \frac{\pi}{4}\sqrt{N}$
- Over-iterating **decreases** success probability
- Must stop at the right time!

Unlike classical search, “trying harder” can make Grover’s **worse** — a uniquely quantum phenomenon.

Shor's Factoring Algorithm

Peter Shor, 1994

Problem: Given $N = p \times q$ (product of two primes), find p and q .

Classical (best known)

General Number Field Sieve
Sub-exponential: $\approx e^{O(n^{1/3})}$

RSA-2048: would take billions of years on best hardware.

Quantum (Shor's)

Polynomial time: $O(n^3)$

RSA-2048: feasible on a large enough quantum computer.

Exponential speedup!

This is why quantum computing matters for cryptography.

Shor's: The Key Idea

Factoring = Period Finding

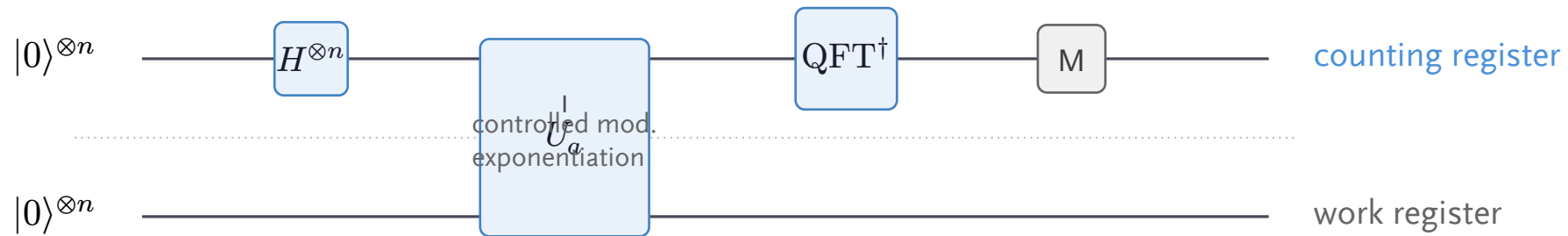
If you can find the period of $f(x) = a^x \bmod N$,
you can factor N efficiently (classical post-processing).

The algorithm:

1. Pick random $a < N$, compute $\gcd(a, N)$ — if not 1, done!
2. Use **Quantum Fourier Transform** to find the period r of $a^x \bmod N$
3. If r is even: compute $\gcd(a^{r/2} \pm 1, N) \rightarrow$ gives factors!
4. Repeat if needed (succeeds with high probability)

The quantum part is only step 2 — the Quantum Fourier Transform (QFT).
Everything else is classical number theory!

Shor's: Circuit — quantum period finding subroutine



$H^{\otimes n}$: creates superposition of all x values in counting register

U_a : computes $a^x \bmod N$ in the work register, controlled by x

$\text{QFT}^\dagger$: extracts period r from the interference pattern; then measure

Quantum Fourier Transform: Intuition

Classical Fourier Transform:

Takes a signal $\rightarrow$ decomposes into component frequencies.

Audio signal $\rightarrow$ “which notes are playing?”

Quantum Fourier Transform:

Takes quantum amplitudes $\rightarrow$ extracts their periodicity.

Encodes the period into measurable qubits.

Why it is fast on a quantum computer:

- Classical FFT on N points: $O(N \log N)$
- QFT on n qubits ($N = 2^n$): $O(n^2)$

QFT is exponentially faster because it works on amplitudes directly — no need to read all N values.

Caveat: you cannot read out the full Fourier transform — only sample from it. But for period finding, that is enough!

Comparing the Algorithms

Algorithm	Classical	Quantum	Speedup
Deutsch–Jozsa	$O(2^n)$	$O(1)$	Exponential
Grover's Search	$O(N)$	$O(\sqrt{N})$	Quadratic
Shor's Factoring	$O(e^{n^{1/3}})$	$O(n^3)$	Super-polynomial

Common pattern:

1. Prepare superposition
2. Query / compute in parallel
3. Use interference to extract answer

Important caveats:

- Quantum computers are NOT faster at everything
- Speedups require structure in the problem
- Grover's quadratic speedup is provably optimal

Part III: The NISQ Era

Hamiltonians, Variational Algorithms & Error Mitigation

The NISQ Era

NISQ = Noisy Intermediate-Scale Quantum (coined by John Preskill, 2018)

Today's quantum computers:

- 50–1000+ qubits (but noisy)
- No full error correction yet
- Gates have 0.1–1% error rates
- Limited circuit depth (100s of gates)

Shor's needs: 4000+ logical qubits
(millions of physical qubits)

We are not there yet!

NISQ strategy:

Use short circuits that can survive noise, combined with classical optimization.

Hybrid quantum-classical algorithms

Key idea: let the quantum computer prepare quantum states, and let the classical computer handle optimization.

What is a Hamiltonian?

The central object in quantum physics:

A Hamiltonian H is a matrix that encodes the **total energy** of a quantum system.

- Its **eigenvalues** = possible energy levels
- Its **eigenvectors** = corresponding states
- The smallest eigenvalue = **ground state energy**

$$H |\psi_n\rangle = E_n |\psi_n\rangle$$

“Apply H to a state $\rightarrow$ get its energy times the state”

Why it matters for quantum algorithms:

- **Chemistry:** H describes a molecule — electrons, nuclei, their interactions
- **Optimization:** H encodes the cost function — ground state = optimal solution
- **Time evolution:** H governs how a quantum state changes over time (e^{-iHt})

For n qubits, H is a $2^n \times 2^n$ matrix.

Classically intractable to diagonalize for large n .
This is where quantum computers help!

Hamiltonians on Qubits: Pauli Decomposition

Any qubit Hamiltonian can be written as a sum of **Pauli strings** — tensor products of I, X, Y, Z :

$$H = \sum_i c_i \cdot P_i \quad \text{where each } P_i \in \{I, X, Y, Z\}^{\otimes n}$$

Example (2-qubit H_2 molecule, simplified):

$$H = -0.81 \cdot II + 0.17 \cdot ZI - 0.23 \cdot IZ + 0.12 \cdot ZZ + 0.04 \cdot XX$$

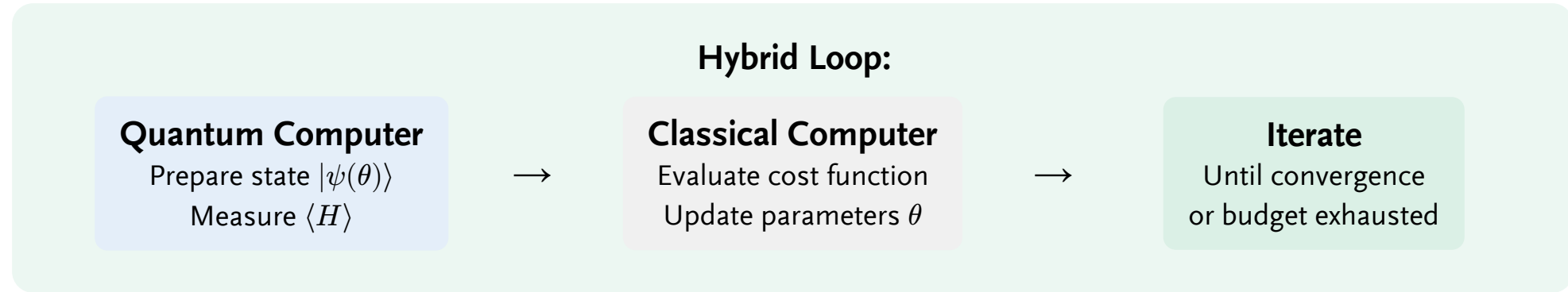
Measuring energy on a quantum computer:

- Cannot measure H directly
- Measure each Pauli term P_i separately
- Average many shots to estimate $\langle P_i \rangle$
- Compute $\langle H \rangle = \sum_i c_i \langle P_i \rangle$

Why this is powerful:

- Number of Pauli terms grows polynomially ($\approx n^4$ for chemistry), not exponentially
- Each term is easy to measure on a quantum computer
- This is the foundation of VQE

The Variational Approach



- The quantum circuit has tunable parameters θ (rotation angles on gates)
- Classical optimizer (e.g., gradient descent) adjusts θ to minimize a cost function
- Analogous to training a neural network, but the “network” is a quantum circuit
- **Variational principle:** $\langle \psi(\theta) | H | \psi(\theta) \rangle \geq E_{\text{ground state}}$ — energy is always an upper bound

VQE: Variational Quantum Eigensolver

Goal: Find the ground-state energy of a molecule.

How VQE works:

1. Encode molecule's Hamiltonian H as Pauli strings
2. Prepare trial state $|\psi(\theta)\rangle$ (ansatz circuit)
3. Measure energy: $E(\theta) = \langle \psi(\theta) | H | \psi(\theta) \rangle$
4. Classical optimizer updates θ
5. Repeat until E converges

Variational principle guarantees: $E(\theta) \geq E_{\text{ground state}}$

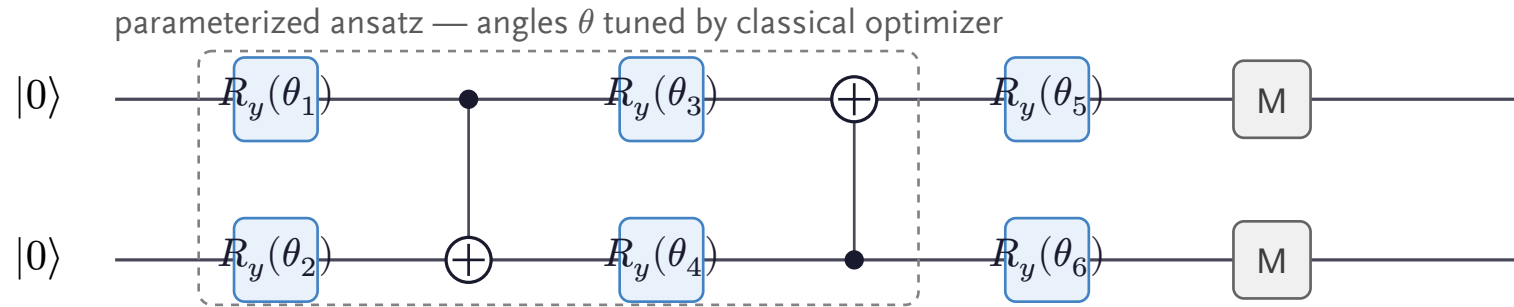
Why it matters:

- Drug discovery: molecular binding
- Materials science: superconductors
- Chemistry: catalyst design

Simulating molecules with n electrons costs $O(2^n)$ classically — exponential!

Quantum computers naturally simulate quantum systems (Feynman's dream, 1982).

VQE: Ansatz Circuit — example for 2 qubits, 2 layers



$R_y(\theta)$ rotation gates
give tunable single-qubit
operations

CNOT gates create
entanglement between
qubits

Deeper circuits are more
expressive but accumulate
more noise

QAOA: Quantum Approximate Optimization

Farhi, Goldstone, Gutmann (2014)

Goal: Find approximate solutions to combinatorial optimization problems.

Example problems:

- Max-Cut: partition graph to maximize cut edges
- Traveling salesman
- Portfolio optimization
- Scheduling

These are **NP-hard** — no known efficient classical solutions.

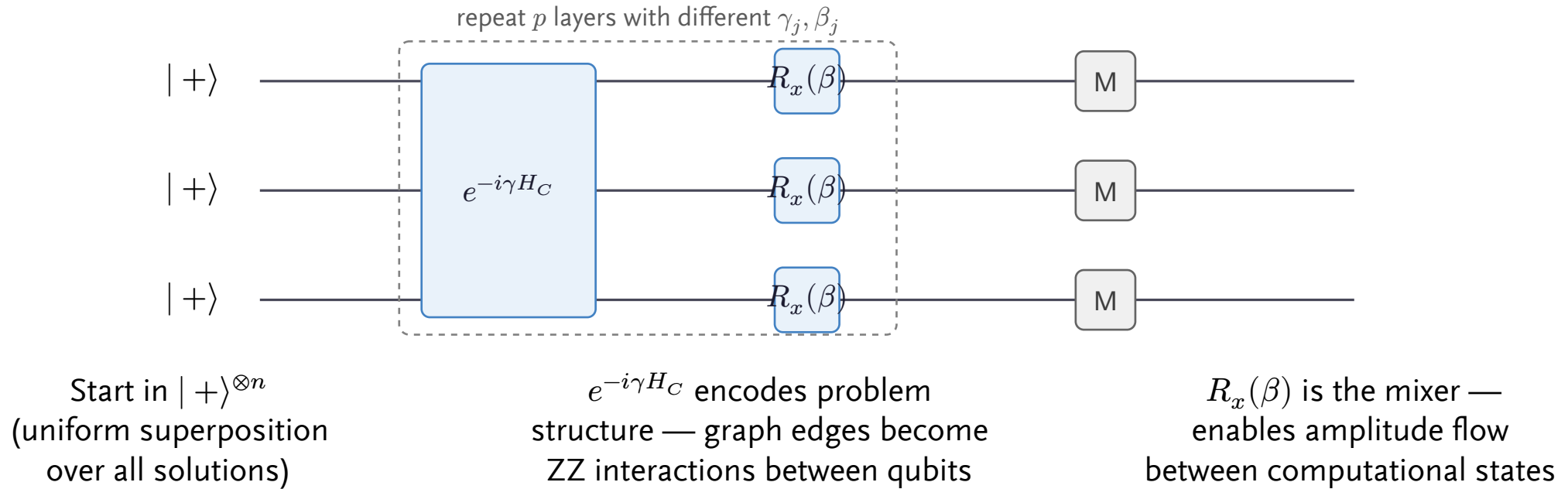
QAOA approach:

1. Encode problem as cost Hamiltonian H_C
2. Alternate between:
 - $e^{-i\gamma H_C}$ — “explore solutions”
 - $e^{-i\beta H_M}$ — “mix amplitudes”
3. Optimize angles γ, β classically
4. Sample from the resulting state

More layers (p) → better approximation
(but deeper circuit → more noise)

Open question: Can QAOA outperform classical heuristics? Active research!

QAOA: Circuit — 1 QAOA layer for 3-qubit Max-Cut



Error Mitigation I: Zero-Noise Extrapolation

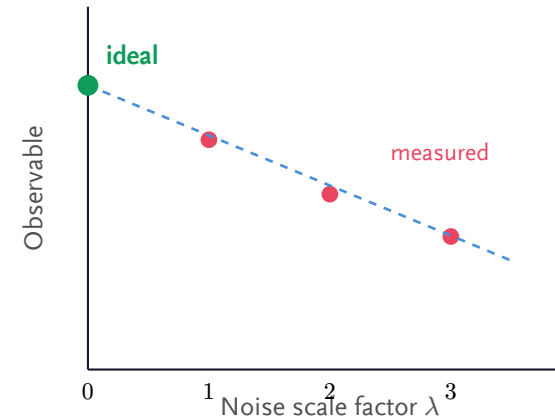
Full error correction requires millions of qubits. Error mitigation extracts better answers from noisy hardware today.

The idea:

1. Run the circuit at the **base noise level**
2. Artificially **increase** noise (e.g., insert identity gate pairs that accumulate errors)
3. Collect results at multiple noise levels
4. **Extrapolate** back to the zero-noise limit

“What would the answer be if there were no noise at all?”

Extrapolation sketch:



Pros: simple to implement, no noise model needed

Cons: assumes noise scales smoothly; needs many shots

Error Mitigation II: Probabilistic Error Cancellation

The idea:

1. **Characterize** the noise channel of each gate (via quantum process tomography)
2. Express the **ideal gate** as a linear combination of noisy operations:

$$\mathcal{G}_{\text{ideal}} = \sum_i \eta_i \mathcal{G}_{\text{noisy}}^{(i)}$$

3. Run many circuits, each sampled from this combination, with signed weights η_i
4. The weighted average of outcomes converges to the ideal (noiseless) expectation value

“Undo the noise mathematically by sampling its inverse.”

Key properties:

Exact in principle: converges to the true noiseless answer (unlike ZNE which is an approximation).

The catch — sampling overhead:

The number of shots needed grows as:

$$\text{shots} \propto \gamma^{2d}$$

where $\gamma \geq 1$ depends on the noise strength and d is the circuit depth.

Exponential cost in depth makes PEC practical only for **shallow circuits** — but that is exactly what NISQ devices run!

Error Mitigation III: Measurement Error Mitigation

The idea:

1. Prepare each computational basis state ($|00\rangle, |01\rangle, |10\rangle, |11\rangle$, etc.)
2. Measure many times to build a **confusion matrix** M
3. M_{ij} = probability of reading i when the true state is j
4. For real data: apply M^{-1} to the measured distribution to recover the corrected distribution

“Correct the detector, not the computation.”

Example confusion matrix (2 qubits):

$$M = \begin{pmatrix} 0.95 & 0.03 & 0.02 & 0.01 \\ 0.02 & 0.93 & 0.01 & 0.03 \\ 0.02 & 0.02 & 0.94 & 0.02 \\ 0.01 & 0.02 & 0.03 & 0.94 \end{pmatrix}$$

Diagonal ≈ 1 means good readout fidelity.

Off-diagonal elements = misidentification rates.

Pros: simple, often the biggest error source

Cons: only fixes readout errors, not gate errors; calibration matrix size is $2^n \times 2^n$ (mitigated via tensor product approximations for large n)

The NISQ Landscape (2024–2026)

Major platforms:

Company	Qubits
IBM	1000+ (superconducting)
Google	70+ (superconducting)
IonQ	36+ (trapped ion)
Quantinuum	56+ (trapped ion)
QuEra	256+ (neutral atom)

Key challenges:

- Error rates still too high for fault-tolerant computation
- Qubit connectivity limitations
- Short coherence times
- Scaling is hard

Milestone (2023):

First demonstrations of quantum error correction below threshold (Google, Quantinuum)

Wrap-Up

Summary

What We Covered Today

Foundations

- Qubits & superposition
- Quantum gates (X, H, CNOT)
- Entanglement
- Interference as the source of speedup

Fault-Tolerant Algorithms

- Deutsch–Jozsa (exponential speedup)
- Grover's search ($\sqrt{N}$ speedup)
- Shor's factoring (breaks RSA)

NISQ Algorithms

- Hamiltonians & Pauli decomposition
- VQE for quantum chemistry
- QAOA for optimization
- Error mitigation (ZNE, PEC, readout)

Key Takeaways

- Quantum is not universally faster
- Speedups need problem structure
- NISQ era: noisy but progressing
- Error mitigation bridges the gap

Thank You!

Questions?

“Nature isn’t classical, dammit, and if you want
to make a simulation of nature, you’d better make it
quantum mechanical.”
— Richard Feynman, 1982